



# CS1002 – Programming Fundamentals

Lecture # 08  
Tuesday, September 13, 2022  
FALL 2022  
FAST – NUCES, Faisalabad Campus

**Rizwan Ul Haq**

# Basic Components of C++ Program

# The Evolution of Programming Languages

- Early computers were programmed in machine language
- To calculate wages = rates \* hours in machine language:

```
100100 010001  //Load rates
100110 010010  //Multiply
100010 010011  //Store in wages
```

## The Evolution of Programming Languages (cont'd.)

- Assembly language instructions are mnemonic
- **Assembler:** Translates a program written in assembly language into machine language

TABLE 1-2 Examples of Instructions in Assembly Language and Machine Language

| Assembly Language | Machine Language |
|-------------------|------------------|
| LOAD              | 100100           |
| STOR              | 100010           |
| MULT              | 100110           |
| ADD               | 100101           |
| SUB               | 100011           |

## The Evolution of Programming Languages (cont'd.)

- Using assembly language instructions,  
wages = rates \* hours can be written as:

LOAD rate

MULT hour

STOR wages

## The Evolution of Programming Languages (cont'd.)

- High-level languages include Basic, FORTRAN, COBOL, Pascal, C, C++, C#, and Java
- **Compiler:** Translates a program written in a high-level language to machine language
- The equation  $\text{wages} = \text{rate} \cdot \text{hours}$  can be written in C++ as:  
$$\text{wages} = \text{rate} * \text{hours} ;$$

# Assembly & Machine Language

## Assembly Language

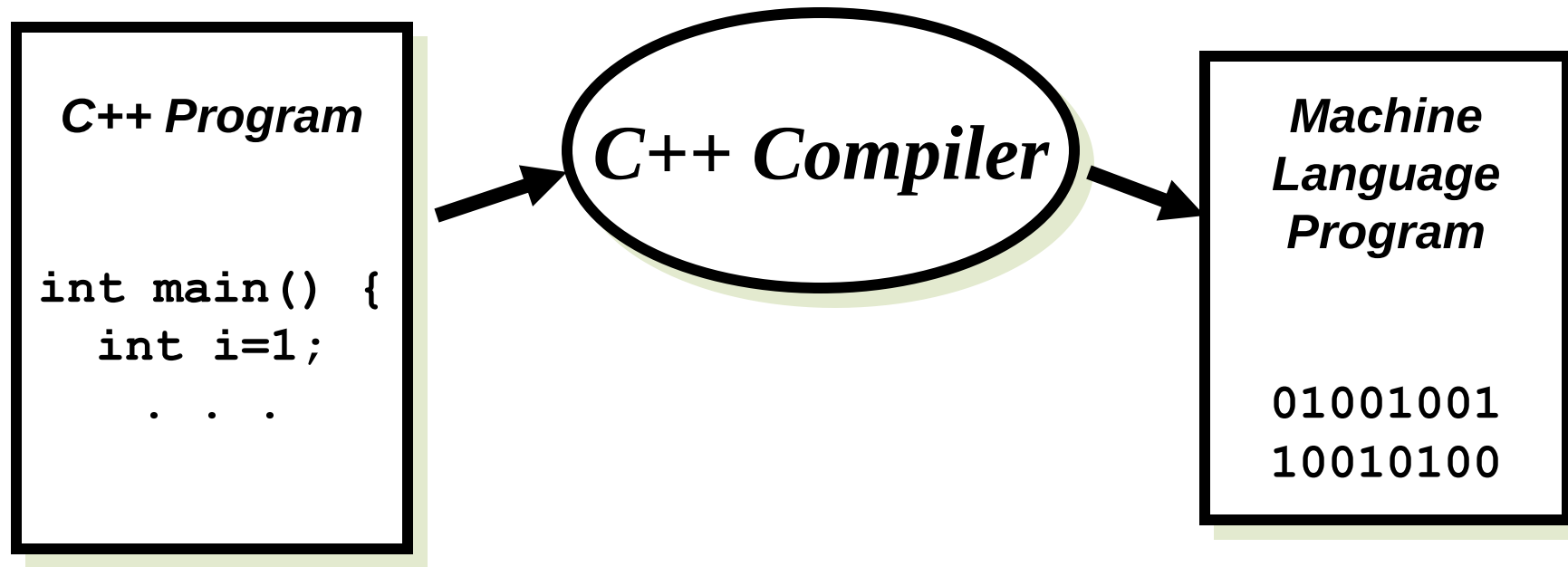
## Machine Language

|      |                   |                                     |
|------|-------------------|-------------------------------------|
|      | ST 1,[801]        | 00100101 11010011                   |
|      | ST 0,[802]        | 00100100 11010100                   |
| TOP: | BEQ [802],10,BOT  | 10001010 01001001 11110000          |
|      | INCR [802]        | 01000100 01010100                   |
|      | MUL [801],2,[803] | 01001000 10100111 10100011          |
|      | ST [803],[801]    | 11100101 10101011 00000010 00101001 |
|      | JMP TOP           | 11010101                            |
|      |                   | 11010100 10101000                   |
| BOT: | LD A,[801]        | 10010001 01000100                   |
|      | CALL PRINT        |                                     |

# Equivalent C/C++ program

```
set memory[801] to hold 00000001 ..... x=1;
set memory[802] to hold 00000000 ..... i=0;
if memory[802] = 10 jump to instruction #8 ..... while (i!=10) {
increment memory[802] ..... i = i+1;
set memory[803] to 2 times memory[801] } x = x*2;
put memory[803] in to memory[801] }
jump to instruction #3 .....
print memory[801] ..... printf("%d",x);
```

# Compiler



Created with text editor or  
development environment

# Processing a C++ Program

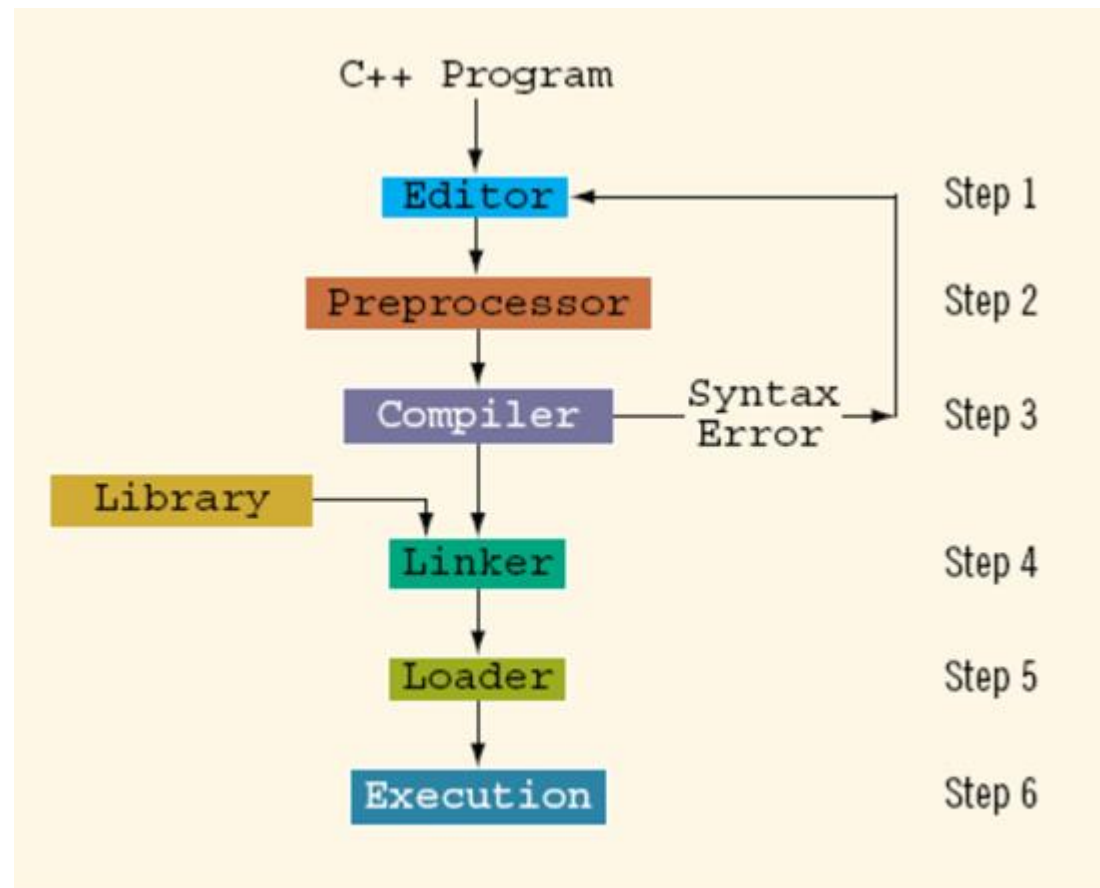


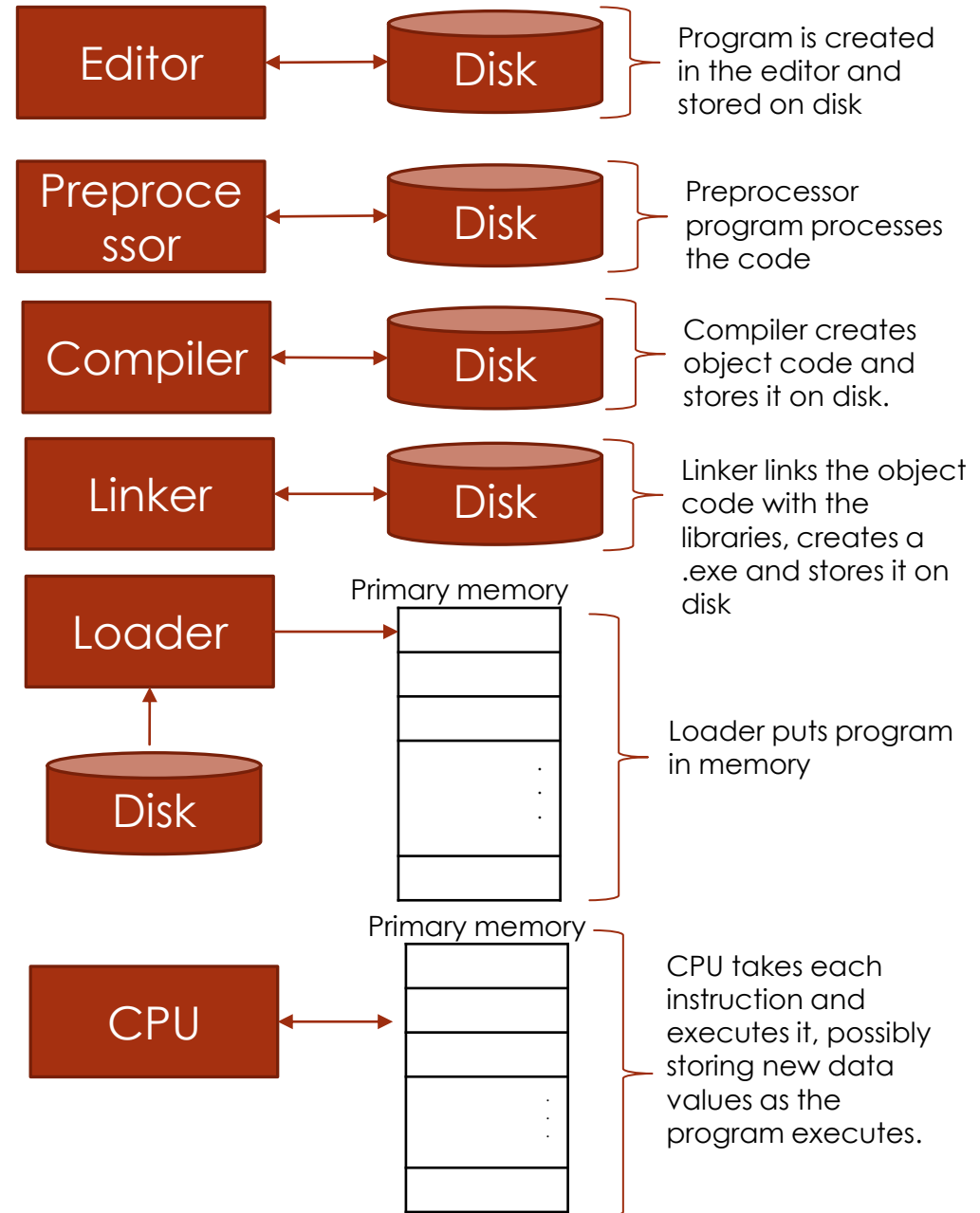
FIGURE 1-3 Processing a C++ program

# Basics of a Typical C++ Environment

12

Phases of C++ Programs:

1. Edit
2. Preprocess
3. Compile
4. Link
5. Load
6. Execute



# Compilers

- Translate high-level language to machine language
- Check that the program obeys the rules
- **Source code**
  - The original program in a high level language
- **Object code**
  - The translated version in machine language

# Linkers

- Some programs we use are already compiled
  - Their object code is available for us to use
  - **For example:** Input and output routines
- A **Linker** combines
  - The object code for the programs we write
  - and**
  - The object code for the pre-compiled routines (**of SDK**)
  - into**
  - The machine language program the CPU can run
- **Loader:**
  - Loads executable program into main memory
- The last step is to execute the program

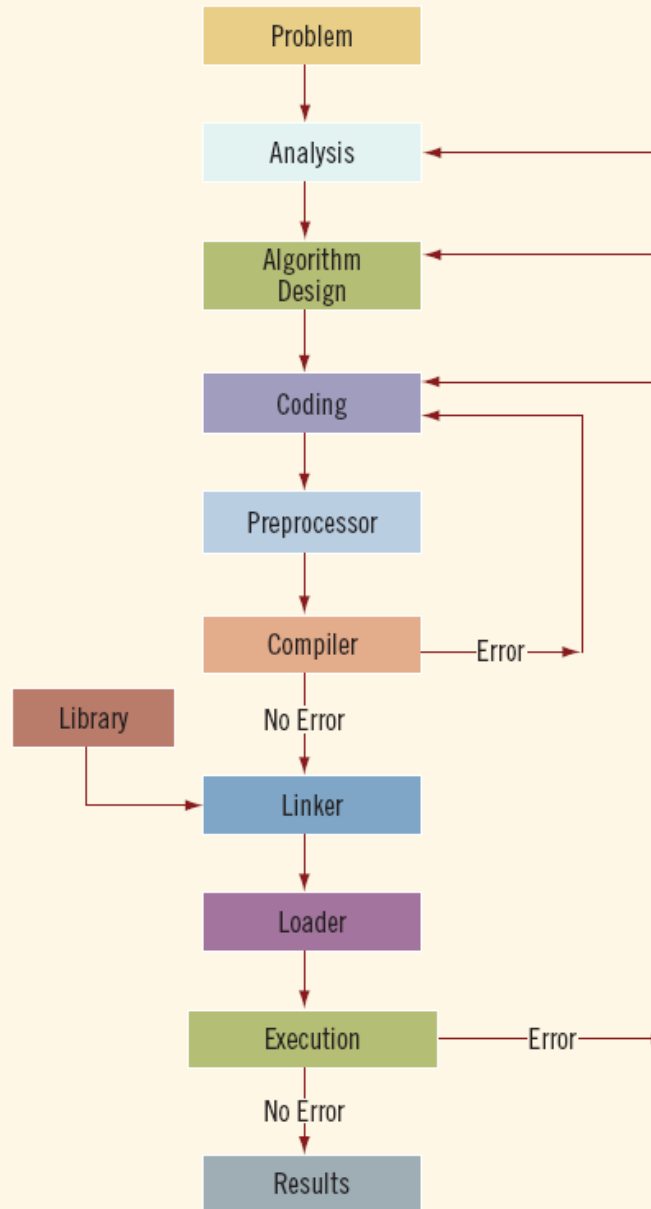


FIGURE 1-4 Problem analysis-coding-execution cycle

# History of C and C++

## ➤ History of C

- Evolved from two other programming languages
- BCPL and B: “Typeless” languages
- **Dennis Ritchie (Bell Lab)**: Added typing, other features
- C is a programming language developed in the 1970's alongside the UNIX operating system
- C provides a comprehensive set of features for handling a wide variety of applications, such as systems development and scientific computation
  - 1989: ANSI standard/ ANSI/ISO 9899: 1990

# History of C and C++

## ➤ History of C++

- Early 1980s: **Bjarne Stroustrup** (Bell Lab)
- Provides capabilities for object-oriented programming
  - **Objects**: reusable software components
  - Object-oriented programs

## ➤ “**Building block** approach” to creating programs

- C++ programs are built from pieces called classes and functions
- **C++ standard library**: Rich collections of existing classes and functions

# Structured/OO Programming

## ➤ Structured programming (1960s)

- Disciplined approach to writing programs
- Clear, easy to test and debug, and easy to modify
- e.g. Pascal: 1971: Niklaus Wirth

## ➤ OOP

- “Software reuse”
- “Modularity”
- “Extensible”
- More understandable, better organized and easier to maintain than procedural programming

# Basics of a Typical C++ Environment

## ➤ C++ systems

- Program-development environment
  - Integrated Development Environment (IDE)
- Language
- C++ Standard Library

## ➤ C++ program names extensions

- .cpp (C Plus Plus)
- .c (C)

# The C++ Standard Library

C/C++ programs consist of pieces/modules called functions

- A programmer can create his own functions
  - **Advantage:** the programmer knows exactly how it works
  - **Disadvantage:** time consuming
- Programmers will often use the C/C++ library functions
  - Use these as building blocks
- Avoid re-inventing the wheel
  - If a pre-made function exists, generally best to use it rather than write your own
  - Library functions carefully written, efficient, and portable

## 21 Programming Style

C++ is a free-format language, which means that:

- Extra blanks (spaces) or tabs before or after identifiers/operators are ignored
- Blank lines are ignored by the compiler just like comments
- Code can be indented in any way
- There can be more than one statement on a single line
- A single statement can continue over several lines

## Programming Style (cont. )

In order to improve the readability of your program, use the following conventions:

- Start the program with a **header** that tells what the program does
- Use **meaningful variable names and Camel notation**
- **Document** each variable declaration with a comment telling what the variable is used for
- Place each **executable statement** on a **single line**
- A segment of code is a sequence of executable statements that belong together
  - Use blank lines to separate different segments of code
  - Document each segment of code with a comment telling what the segment does.

# C++ keywords

- Keywords appear in **blue** in Visual C++
- Each keyword has a predefined purpose in the language
- Do not use keywords as variable and constant names!!
- We shall cover most of the following keywords in this class:

bool, break, case, char, const, continue, do, default, double, else, extern, false, float, for, if, int, long, namespace, return, short, static, struct, switch, typedef, true, unsigned, void, while

# Structure of a C++ Program

A C++ program is a collection of definitions and declarations:

- data type definitions
- global data declarations
- function definitions (subroutines)
- class definitions
- a special function called
  - **main()** (where the action starts)

# General form of a C++ program

```
// Program description
#include directives
global declarations
int main()
{
    constant declarations
    variable declarations
    executable statements
    return 0;
}
```

# Includes

- The statement: **#include <iostream>** inserts the contents of the file **iostream** inside your file before the compiler starts
- Definitions that allow your program to use the functions and classes that make up the standard C++ library are in these files.
- You can include your own file(s):

```
#include "myfile.h"
```

# C++ compiler directives

- Compiler directives appear in **blue** in Visual C++
- The **#include** directive tells the compiler to include some already existing C++ code in your program
- The included file is then linked with the program
- There are two forms of #include statements:

**#include** <iostream> //for pre-defined files

**#include** "my\_lib.h" //for user-defined files

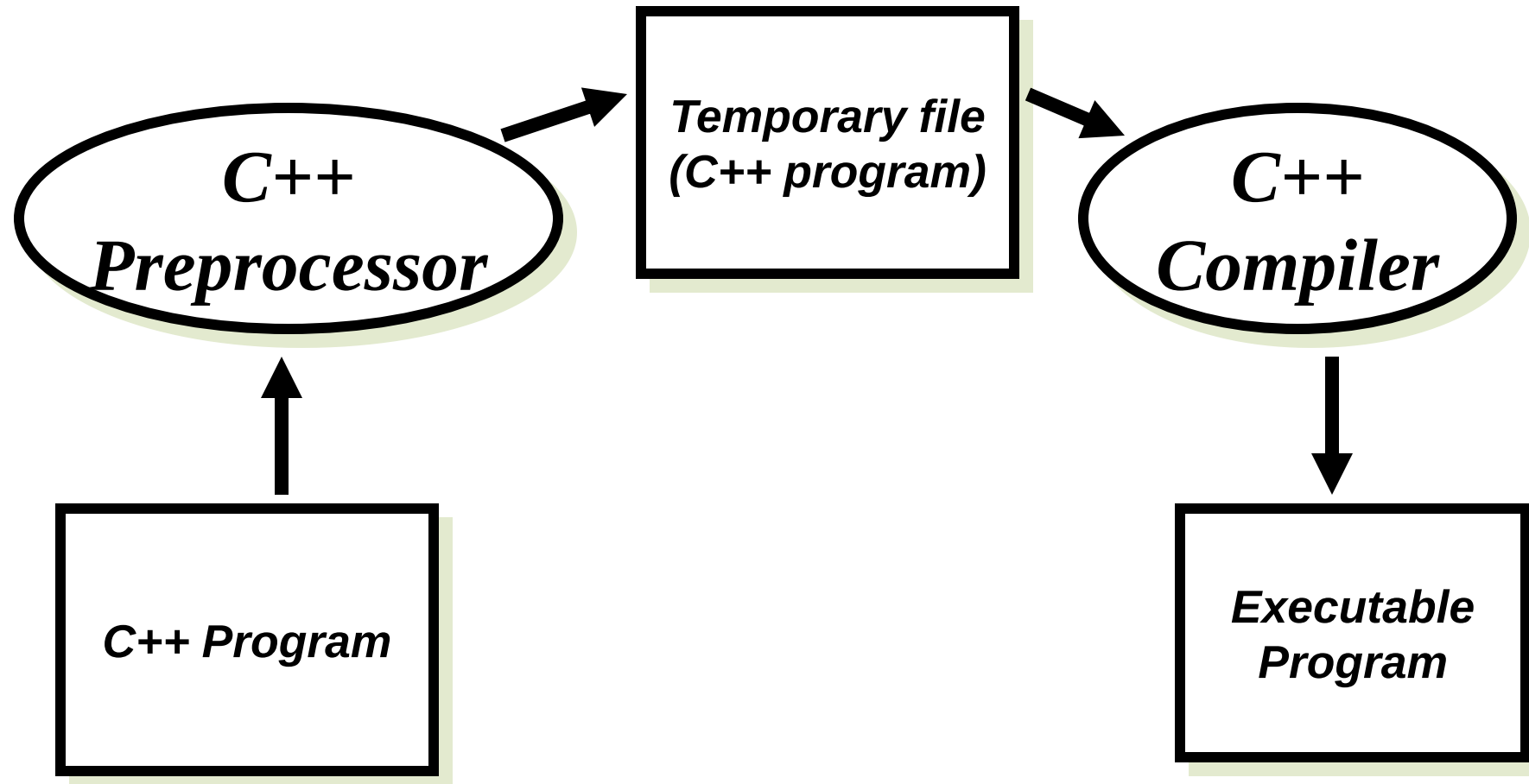
# The Preprocessor

- Lines that start with the character '#' are special instructions to a preprocessor
- The preprocessor can replace the line with something else:
  - **include:** replaced with contents of a file
- Other *directives* tell the preprocessor to look for patterns in the program and do some fancy processing

# C++ Preprocessor

- C++ Compilers **automatically invoke a preprocessor** that takes care of #include statements and some other special directives
- Definitions that **allow your program** to use the functions and classes that make up the standard C++ library are in these files
- You don't need to do anything special to run the preprocessor - it happens automatically

# Preprocessing



# Some common include statements

- Basic I/O: `iostream.h`
  - Provides functionality of input and output
- I/O manipulation: `iomanip.h`
  - Formats the input and output
- Standard Library: `stdlib.h`
  - Functions for memory allocation, process control, conversion etc.
- Time and Date support: `time.h`
  - Functionality of time manipulation
- Mathematics support: `math.h`
  - Functionality of basic mathematical functions

# Questions

